I dati — il registro, chiave per chiave

Questo documento spiega **come sono fatti i dati** di «La mia settimana»: come si compone ogni informazione salvata,
chi la scrive, chi la legge e che cosa ne fa. Il README racconta l'app; questo è il glossario dei dati.

Tutto il codice citato sta in `core.js` (il nucleo) e in `js/` (la pagina).

Indice

1. In breve
2. Il record: chiave e valore
3. Come si compone una chiave
4. Glossario delle chiavi
5. Vuoto, togliere, cancellare
6. Come si scrive: dal tocco al registro
7. Come si legge: dal registro allo schermo
8. L'unione tra dispositivi
9. Il file di sincronizzazione
10. Le altre chiavi del browser (fuori dal registro)
11. Chiavi sconosciute, e come aggiungerne una

1. In breve

- I dati sono un **registro piatto**: un elenco di coppie *chiave* → *record*, senza tabelle né annidamenti.
- Ogni chiave è **una sola informazione** («Sviluppo è fatto mercoledì della settimana del 12 ottobre»).
- Ogni record porta con sé il **valore**, il **momento** della modifica e il **dispositivo** che l'ha fatta.
- Le chiavi le scrive **solo il nucleo** (`core.js`), con le sue operazioni. La pagina chiede, il nucleo scrive.
- Quello che vedi sullo schermo **non è salvato**: si ricalcola ogni volta dal registro (la *vista*).
- Niente si cancella davvero: togliere vuol dire scrivere un valore vuoto (`false`, `''`, `null`), con il suo momento.

2. Il record: chiave e valore

Un elemento del registro:

```
"s/2026-10-12/a/sviluppo/f/2": { v: true, t: 1760436000123, d: "d4f2a91bc" }
```

Parte	Che cos'è	Esempio
chiave	*dove* sta l'informazione: un percorso di segmenti separati da `/` (§3)	`s/2026-10-12/a/sviluppo/f/2`
`v`	il **valore**: vero/falso, un numero, un testo, `null`	`true`

Parte	Che cos'è	Esempio
`t`	il **momento** della modifica, in millisecondi dal 1970 (come `Date.now()`)	`1760436000123`
`d`	il **dispositivo** che l'ha scritta: un nome casuale, uno per browser	`d4f2a91bc`

`t` e `d` servono solo all'**unione** tra dispositivi (§8): quando due copie hanno la stessa chiave, vince il record più recente. Non compaiono mai sullo schermo.

Perché piatto. Se i dati fossero un grande oggetto annidato («la settimana contiene le attività, che contengono
le spunte»), unire due copie vorrebbe dire confrontare strutture intere, e una modifica piccola su un dispositivo
potrebbe cancellarne una piccola sull'altro. Con un'informazione per chiave, ogni modifica è indipendente dalle altre:
la spunta di lunedì e quella di martedì non si toccano mai. Nei libri lo schema si chiama *CRDT* (un registro "ultimo scritto vince" per ogni chiave).

3. Come si compone una chiave

Una chiave è fatta di segmenti. Il **primo** dice di che cosa parla:

Primo segmento	Parla di	Forma
`v`	una **voce** della libreria (vale in tutte le settimane)	`v/<id>/<campo>`
`s`	una **settimana**	`s/<lunedì>/…`
`i`	un'**impostazione** che vale su tutti i dispositivi	`i/<nome>`

I pezzi che si ripetono:

- `<id>` **— l'identificativo di una voce.** Si ricava dal nome con `slug()`: minuscole, senza accenti, con i trattini al posto degli spazi. «Città nuova» → `citta-nuova`. Così la stessa voce, creata su due dispositivi prima
  di unirli, è una voce sola. Se l'identificativo è già di una voce con un altro nome (per esempio una voce rinominata), `idPer()` aggiunge `-2`, `-3`… Un nome senza lettere latine (solo emoji) diventa `x` più un'impronta
  del nome. Forma ammessa: `[a-z0-9-]`, da 1 a 48 caratteri. **L'identificativo non cambia mai**, nemmeno
  rinominando: il nome sta nella chiave `v/<id>/nome`.
- `<lunedì>` **— la settimana.** La data del suo lunedì, `AAAA-MM-GG`. Deve essere un lunedì vero: `valida()` scarta
  una chiave `s/` con un'altra data.
- `<giorno>` **— il giorno della settimana.** Una cifra da `0` (lunedì) a `6` (domenica).
- `a/<id>` dentro una settimana: «quella voce, in quella settimana».

Esempio letto da sinistra a destra:

```
s / 2026-10-12 / a / sviluppo / f / 2
|   |            |   |          |   └─ mercoledì (0 = lunedì)
|   |            |   |          └──── f = fatto
|   |            |   └─────────────── la voce «sviluppo»
|   |            └─────────────────── a = un'attività di quella settimana
|   └──────────────────────────────── la settimana del lunedì 12 ottobre 2026
└──────────────────────────────────── s = settimana
```

4. Glossario delle chiavi

Per ogni chiave: la forma, i valori ammessi, che cosa vuol dire, **chi la scrive** (l'operazione del nucleo e da dove la chiama la pagina) e **chi la legge** (la funzione del nucleo e dove si vede). I valori ammessi sono quelli che controlla `valida()` (l'elenco `SCHEMI` in `core.js`): un record con un valore diverso viene scartato.

Voci della libreria

`v/<id>/nome` — il nome della voce

- **Valore:** testo, da 1 a 60 caratteri, già ripulito (spazi in eccesso tolti); mai «Riposo» o «Energia», che sono nomi riservati alle righe in fondo alla tabella.
- **Esempio:** `v/sviluppo/nome` → `"Sviluppo"`
- **Scrive:** `aggiungi` e `nuovaFaccenda`, solo per una voce nuova (riprendere una voce cancellata non la rinomina);
  `rinomina`. Dalla pagina: il campo «Aggiungi», i suggerimenti, il foglio della faccenda in Casa (`casa.js`), la
  libreria e il foglio «Rinomina» (`libreria.js`).
- **Legge:** `vista()` costruisce la libreria (`state.libreria`: `{ id, name }`). Una voce senza nome non esiste: le
  sue chiavi di settimana restano nel registro ma non si mostrano. Il nome vale in **tutte** le settimane: rinominare
  cambia anche il passato.

`v/<id>/del` — la voce è stata cancellata dalla libreria

- **Valore:** `true` / `false`.
- **Scrive:** `cancella` (solo una voce mai usata: né in una lista né con una spunta), dal pulsante «Cancella» della
  libreria. `aggiungi` e `nuovaFaccenda` scrivono `false`, cioè la riprendono.
- **Legge:** `vista()`: una voce con `del` = `true` sparisce dalla libreria, **a meno che** non sia usata da qualche parte. Se un dispositivo la cancella e l'altro, prima di unire, la usa, vince l'uso.

`v/<id>/cad` — la cadenza: la voce è una faccenda

- **Valore:** un numero intero di giorni da 1 a 365, oppure `false` (nessuna cadenza).
- **Esempio:** `v/bagno/cad` → `14` (ogni 14 giorni)
- **Scrive:** `nuovaFaccenda` e `cadenza`, dal foglio della faccenda in Casa (`casa.js`); `cadenza(id, null)` scrive

`false` («Togli la cadenza»).

- **Legge:**
  - `faccende()`: chi è faccenda, l'ultima volta, il prossimo giro, il calore. Lo usano Casa, il promemoria e le
    schede della settimana (`testi.js` per le parole).
  - `vista()`: una voce con cadenza in una settimana ha `tipo: 'faccenda'`, e i giorni in cui si vede sono quelli non
    coperti da una spunta vicina (§7).
  - `aggiungi`: una faccenda va sempre sulla settimana intera, anche se la pagina indica un giorno.

`v/<id>/salto` — «Salto questo giro»

- **Valore:** una data `AAAA-MM-GG`, oppure `''` (nessun salto).
- **Esempio:** `v/bagno/salto` → `"2026-10-22"`
- **Scrive:** `salta(id, oggi)` dal pulsante «Salto questo giro» in Casa e nel promemoria; `salta(id, null)` scrive
  `''` («Annulla»).
- **Legge:** `faccende()`. Il salto conta solo se è dopo l'ultima spunta e non nel futuro: il conteggio riparte da
  quel giorno, ma «l'ultima volta» resta vera.

Settimane

`s/<lunedì>/m/<id>` — la voce è nella lista della settimana

- **Valore:** `true` / `false`.
- **Esempio:** `s/2026-10-12/m/sviluppo` → `true`
- **Scrive:** `aggiungi` (`true`). Dalla pagina: «Aggiungi» e i suggerimenti (`azioni.js`), «Aggiungi» del promemoria
  (`promemoria.js`) e di Casa (`casa.js`). `togli` scrive `false` (la × della tabella e «Togli dalla settimana»); `togliGiorno` scrive `false` quando un'attività non resta in nessun giorno.
- **Legge:** `vista()`: le righe della settimana (`state.weeks[lunedì].activities`). **L'ordine delle righe è il
  momento `t` di questa chiave**: le attività compaiono nell'ordine in cui le hai aggiunte.

`s/<lunedì>/a/<id>/f/<giorno>` — fatto quel giorno

- **Valore:** `true` / `false`.
- **Esempio:** `s/2026-10-12/a/sviluppo/f/2` → `true` (fatto mercoledì)
- **Scrive:** `spunta`, dalle caselle della tabella e delle schede (`azioni.js`). Si può spuntare qualunque giorno,
  anche fuori programma.
- **Legge:**
  - `vista()`: le spunte della settimana (`done`), e i giorni in cui un'attività si vede (§7).
  - `faccende()`: **l'ultima volta** di una faccenda è la spunta più recente, in **qualunque** settimana, anche in una
    settimana da cui la voce è stata tolta. Una spunta nel futuro (orologio sbagliato) non conta.

    - `vista()` per le faccende: ogni spunta copre i giorni vicini (§7).
    - `cancella`: una voce con spunte non si cancella.
- **Nota:** togliere una voce dalla settimana **non** tocca le sue spunte. Restano nel registro come storico e ricompaiono se la voce torna in quella settimana.

`s/<lunedì>/a/<id>/g/<giorno>` — in programma quel giorno

- **Valore:** `true` / `false`.
- **Esempio:** `s/2026-10-12/a/sviluppo/g/1` → `true` (Sviluppo è in programma martedì)
- **Scrive:** `aggiungi(lunedì, nome, giorno)` (`true`): sul telefono il giorno che stai guardando, sul computer
  oggi (`giornoPerAggiungere` in `settimana.js`). `togliGiorno` scrive `false` («Togli da questo giorno», sulle schede
  del telefono); `togli` scrive `false` in tutti e sette i giorni.
- **Legge:** `vista()`: un'attività che ha almeno una chiave `g` ha `tipo: 'giorni'` e si vede nei giorni in programma e in quelli in cui è segnata. Un'attività **senza nessuna** chiave `g` (aggiunta prima che i giorni esistessero) ha `tipo: 'settimana'` e vale per tutti e sette. Le faccende ignorano `g`.
- **Regola:** un giorno in cui l'attività è segnata non si toglie (`togliGiorno` risponde `fatta`): la spunta è storia.

`s/<lunedì>/a/<id>/det` — il dettaglio della settimana

- **Valore:** testo, fino a 200 caratteri (`''` = nessun dettaglio).
- **Esempio:** `s/2026-10-12/a/sviluppo/det` → `"Programma A"`
- **Scrive:** `dettaglio`, dal campo sotto il nome (tabella e schede, `azioni.js`).
- **Legge:** `vista()` (`activities[].dettaglio`). Ogni settimana ha il suo: la voce è «Sviluppo», il dettaglio
  dice cosa nello specifico quella settimana.

`s/<lunedì>/r/<giorno>` — giorno di riposo

- **Valore:** `true` / `false`.
- **Scrive:** `riposo`, dalla riga «Giorno di riposo» della tabella e dall'interruttore della vista a un giorno.
- **Legge:** `vista()` (`rest`). La pagina: tinta del riposo sulla colonna o sulla giornata; con il riposo oggi, le faccende perdono il calore (`riposoOggi` in `base.js`).

`s/<lunedì>/e/<giorno>` — energia del giorno

- **Valore:** `"pesa"`, `"ok"`, `"leggero"` oppure `null` (non indicata).
- **Scrive:** `energia`, dai tre puntini della tabella e dai tre segmenti del telefono. Un secondo tocco sullo stesso
  livello scrive `null`.
- **Legge:** `vista()` (`energy`). La pagina: i puntini, il colore del giorno nella striscia, il CSV.

Impostazioni condivise

`i/intensita` — quanto si colorano le faccende oltre la cadenza

- **Valore:** `"nessuna"`, `"morbida"`, `"decisa"`.
- **Scrive:** `intensita`, dal menu («Colore delle faccende», `guscio.js`).
- **Legge:** `vista()` (`intensita`, `'morbida'` se nessuno l'ha mai scelta). `disegna()` in `base.js` la mette su
  `<html data-intensita>`, e `stile.css` fa il resto. Sta nel registro perché vale su **tutti** i dispositivi; il tema
  chiaro/scuro invece no (§10).

5. Vuoto, togliere, cancellare

- **Un valore vuoto equivale a una chiave assente.** `false`, `''` e `null` sono "vuoti": `scrivi()` non registra un
  vuoto se la chiave non c'è già. Così il registro contiene solo ciò che hai davvero fatto.
- **Togliere non cancella la chiave.** Togliere una spunta scrive `false`, togliere una voce dalla settimana scrive
  `m` = `false`. Se la chiave venisse eliminata, il file vecchio dell'altro dispositivo la rimetterebbe, perché per l'unione una chiave assente è "niente da confrontare". Un `false` con il suo momento invece vince sul `true` più
  vecchio.
- **Cancellare una voce** (`v/<id>/del` = `true`) è possibile solo se non è mai stata usata: una voce usata fa parte
  dello storico.

6. Come si scrive: dal tocco al registro

```
 tocco ──► js/azioni.js (o casa.js, promemoria.js, libreria.js, guscio.js)
             │  db.spunta(lunedì, id, giorno, true)
             ▼
           core.js: l'operazione controlla che abbia senso (giorno 0-6, voce nella
settimana…)
             │  se no: { ok: false, errore: 'non-trovata' }
             │  se sì: scrivi(chiave, valore)
             ▼
           scrivi(): se il valore non cambia, niente; altrimenti
                     reg[chiave] = { v, t: momento, d: questo dispositivo }
             ▼
           esito() in base.js: salva() → localStorage «settimana:registro»;
rileggi() → state = vista
```

Il momento `t`. Non è semplicemente l'ora del dispositivo: `scrivi()` usa il più grande tra l'ora attuale e l'ultimo momento visto più uno. Così l'orologio del registro **non torna mai indietro**: anche su un dispositivo con
l'ora sbagliata, ciò che fai *dopo* un'unione vince su ciò che hai appena unito.

Chi può scrivere. Solo le operazioni del nucleo restituite da `Core.crea()` (`aggiungi`, `togli`, `togliGiorno`, `spunta`, `dettaglio`, `riposo`, `energia`, `rinomina`, `cancella`, `nuovaFaccenda`, `cadenza`, `salta`, `intensita`). La pagina non scrive mai una chiave a mano: così le regole (nomi riservati, doppioni, voci usate)

valgono
sempre, da qualunque punto arrivi la richiesta.

7. Come si legge: dal registro allo schermo

```
 localStorage «settimana:registro»  (oppure un file di sincronizzazione unito)
      │  leggiSalvato() → Core.daFile(): controlla il formato
      ▼
 Core.valida(): tiene solo i record ben formati (§4 e §11)
      ▼
 db = Core.crea({ dispositivo, registro })
      │
      ├─► Core.vista(registro)       → state: libreria, settimane (attività, tipo,
giorni, spunte, riposo, energia),
      │                                  intensità. Ricalcolata a ogni modifica
(rileggi), mai salvata.
      ├─► Core.faccende(reg, oggi)   → ultima volta, prossimo giro, calore: Casa,
promemoria, schede, tabella
      └─► Core.primoUso(reg)         → da quando usi l'app: la sezione Dati, per
l'invito alla copia
      ▼
 disegna() in base.js → la sezione aperta, la libreria del menu, il puntino su
Dati
```

I giorni in cui si vede un'attività (`activities[].giorni` nella vista) si calcolano qui, a ogni lettura:

- `tipo: 'giorni'` (ha chiavi `g`): i giorni con `g` = `true`, più quelli con `f` = `true`. Se non ne resta nessuno, l'attività non compare.
- `tipo: 'faccenda'` (ha `v/<id>/cad`): tutti i giorni, tranne quelli **coperti**. Un giorno è coperto se una spunta
  di quella voce, in qualunque settimana e in un altro giorno, dista meno di una cadenza. Un giorno con la sua spunta
  si vede sempre.
- `tipo: 'settimana'` (nessuna chiave `g`): tutti e sette.

8. L'unione tra dispositivi

`Core.unisci(qui, arrivato)` confronta le due copie **chiave per chiave**:

- una chiave che c'è solo nel file arrivato → si aggiunge;
- una chiave che c'è in tutte e due → vince il record con `t` maggiore; a parità di `t`, quello con `d` maggiore; a
  parità anche di quello, il valore maggiore. È sempre la stessa risposta, su qualunque dispositivo (`vince()`).

Ne seguono tre garanzie, controllate dai test di proprietà in `test/core.test.js`: l'ordine delle unioni non conta,
unire due volte lo stesso file non cambia niente, un file vecchio non toglie niente a chi è andato avanti.

9. Il file di sincronizzazione

Lo stesso formato serve per la memoria del browser (`settimana:registro`) e per il file che si salva e si unisce dalla sezione Dati (`settimana-sync-AAAA-MM-GG.json`). Lo costruisce `Core.perFile()`, lo legge `Core.daFile()`:

```
{
  "formato": "settimana-sync",
  "versione": 1,
  "salvatoIl": "2026-10-14T10:00:00.000Z",
  "dispositivo": "d4f2a91bc",
  "registro": {
    "v/sviluppo/nome": { "v": "Sviluppo", "t": 1760436000123, "d": "d4f2a91bc" },
    "s/2026-10-12/m/sviluppo": { "v": true, "t": 1760436000124, "d": "d4f2a91bc" }
  }
}
```

- `formato` deve essere `settimana-sync`: un qualunque altro file viene rifiutato.
- `salvatoIl` e `dispositivo` sono solo informazioni (la conferma di unione dice quando è stato salvato).
- `registro` passa da `valida()` prima di essere unito.

10. Le altre chiavi del browser (fuori dal registro)

Queste stanno in `localStorage`, ma **non** nel registro: non viaggiano con il file e non si uniscono. Sono comodità
del singolo dispositivo.

Chiave	Valore	Scrive / legge	Perché non è nel registro
`settimana:registro`	il registro, nel formato del §9	`salva()` / `caricaRegistro()` in `base.js`	è il registro stesso
`settimana:registro:illeggibile`	la copia di un registro rovinato	`caricaRegistro()`, una volta	si conserva per recuperarlo a mano
`settimana:dispositivo`	il nome casuale di questo browser (`d…`)	`idDispositivo()` in `base.js`	identifica il dispositivo, che è proprio ciò che non si condivide
`settimana:sync`	`{ salvato: 'AAAA-MM-GG', unito: 'AAAA-MM-GG' }`	`segnaSync()` / `leggiSync()` in `file.js`	quando *questo* dispositivo ha salvato o unito: l'invito alla copia è suo

Chiave	Valore	Scrive / legge	Perché non è nel registro
`settimana:tema`	`"chiaro"` / `"scuro"` (assente = automatico)	menu in `guscio.js`; letta anche da uno script in testa a `index.html`	si può volere il telefono scuro e il computer chiaro
`settimana:sezione`	`"settimana"`, `"casa"`, `"dati"`	`sezioni.js`	l'app riapre dove l'avevi lasciata, su quel dispositivo

11. Chiavi sconosciute, e come aggiungerne una

Chiavi sconosciute. `valida()` conserva anche le chiavi che non conosce, purché ben formate: forma `lettera/segmenti`, valore semplice (vero/falso, numero, testo breve, `null`). Una versione vecchia dell'app che riceve
un file con una chiave nuova (per esempio `…/g/<giorno>`, arrivata con la settimana giorno per giorno) **non la mostra,**
ma non la perde, e la ripassa nel file successivo. Per questo non serve aggiornare tutti i dispositivi nello stesso momento.

Aggiungere una chiave, dal più piccolo al più grande:

1. Scegliere la forma seguendo il §3: di che cosa parla (`v`, `s`, `i`), poi il percorso.
2. Aggiungerla a `SCHEMI` in `core.js`, con la regola del valore.
3. Scriverla solo da un'operazione del nucleo (in `crea()`), con i suoi controlli.
4. Leggerla in `vista()` o in `faccende()`.
5. Metterla tra le modifiche casuali dei test di proprietà (`modificaCasuale` in `test/core.test.js`): sono loro a
   dire se l'unione regge.
6. Aggiornare **questo documento**, la tabella del registro nel README (§6) e le regole in `CLAUDE.md`.